Introduction to Transactions

DBMS Lecture Series

Unit-6

Dr. Pavinder Yadav

Assistant Professor
School of Computer Science
University of Petroleum and Energy Studies
Dehradun

Outline

Table of Contents

What is a *Transaction*?

In DBMS, a transaction is a **logical unit of work** — a sequence of read/write (SQL) operations that transforms the database from one *consistent* state to another.

Formal intuition

A transaction T is an ordered sequence of operations $\{read(x), write(x), \dots\}$ executed atomically.

Why it matters

Either **all** its operations take effect (*commit*) or **none** (*rollback*). No partial effects.

Examples

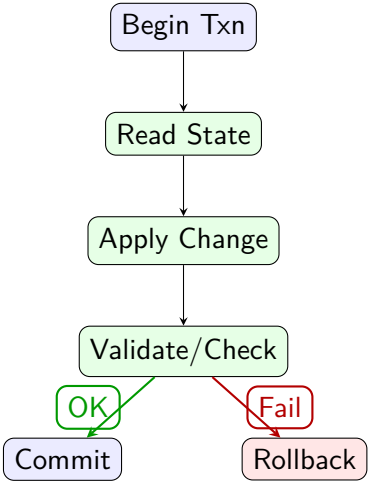
- Bank transfer (A $\rightarrow$ B)
- E-commerce checkout (cart $\rightarrow$ order)
- Seat booking (reserve, pay, ticket)

Kitchen Order Analogy

Chef reads order, cooks dish, plates, serves. Any failure $\Rightarrow$ *discard* and redo. The customer must either get the complete dish or nothing.

Examples

- **Library issue**: check availability $\rightarrow$ issue $\rightarrow$ due-date update
- **Wallet top-up**: debit card $\rightarrow$ credit wallet $\rightarrow$ notify

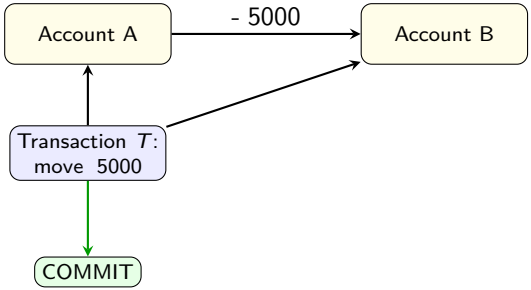


Bank Transfer Example (A → B)

Examples

Transfer 5000 from A to B:

1. Read $bal(A)$
2. $bal(A) \leftarrow bal(A) - 5000$
3. Read $bal(B)$
4. $bal(B) \leftarrow bal(B) + 5000$
5. Write back, then **commit**



Invariant

Total money conserved: $bal(A) + bal(B) = \text{constant}$

If a crash occurs mid-way

We must **rollback** to ensure no partial debit/credit appears.

A Minimal SQL View of a Transaction

Transactional template

```
BEGIN TRANSACTION;

UPDATE accounts SET balance = balance - 5000
WHERE acc_no = 'A';

UPDATE accounts SET balance = balance + 5000
WHERE acc_no = 'B';

COMMIT; -- All-or-nothing boundary
-- ROLLBACK; -- If any step fails
```

Examples

- Use `BEGIN`/`COMMIT`/`ROLLBACK` to mark boundaries.
- *Application logic* ensures invariants (e.g., non-negative balance).

Scenario

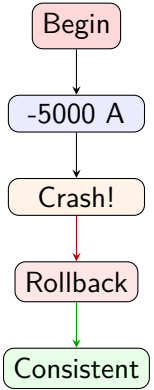
Crash occurs after debiting A but before crediting B.

Without transactions

A is lower by 5000, B unchanged $\Rightarrow$ **inconsistency**.

Examples

With transactions: system **rolls back** to pre-debit state, preserving the invariant.



E-Commerce Checkout as a Transaction

Examples

Steps:

1. Decrement inventory
2. Create order
3. Charge payment
4. Generate invoice/notify

All-or-none ensures no order without payment or vice-versa.

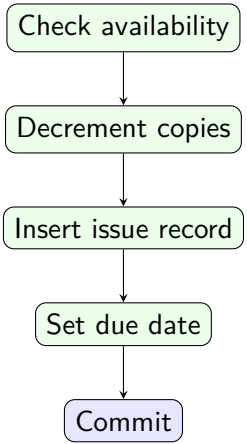
```
BEGIN TRANSACTION;

UPDATE stock SET qty = qty - 1
WHERE sku = 'P123' AND qty > 0;

INSERT INTO orders(user_id, sku, status)
VALUES (42, 'P123', 'CREATED');

-- Call payment service (idempotent)
-- Record payment success/failure

UPDATE orders SET status = 'CONFIRMED'
WHERE order_id = :oid;

COMMIT; -- or ROLLBACK on any failure
```

Business rules (constraints)

- Copy must be available: `copies_available > 0`
- User quota not exceeded: `issued_count` $\leq$ `quota`
- Due date correctly assigned

Consistency

A transaction must **preserve** all integrity constraints.



Summary — Introduction to Transactions

- A transaction is the smallest unit of consistency in a DB.
- **Boundaries**: `BEGIN` → work → `COMMIT/ROLLBACK`.
- **ACID** guarantees correctness amid failures and concurrency.
- Real-world flows: bank transfer, checkout, booking, library issue.

Key mental model

"*All-or-none, preserves rules, plays well with others, and sticks once done.*"

Examples

Next up: **Transaction and System Concepts** — states, failures, and their handling.

Table of Contents

Where Do Transactions Live? The System View

DBMS Execution Environment

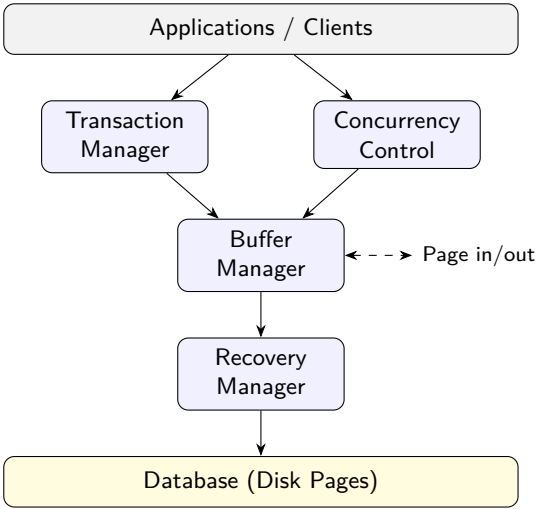
- **Database** (on disk) + **Buffer Manager** (in RAM)
- **Transaction Manager** (IDs, states, begin/commit/abort)
- **Concurrency Control Manager** (isolation: locks/timestamps)
- **Recovery Manager** (atomicity & durability via logs/checkpoints)

Examples

Intuition: The *Transaction Manager* is the coordinator; the *CCM* prevents interference; the *Recovery Manager* ensures "all-or-none" even after crashes.

Roles

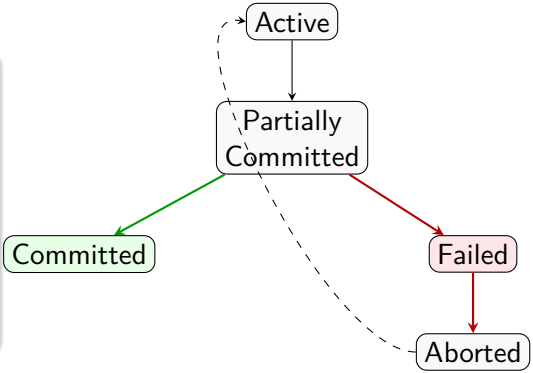
- **TM**: begin/commit/abort; tracks states
- **CCM**: 2PL, timestamps, validation
- **RM**: logging, redo/undo, checkpoints
- **Buffer Mgr**: pages in RAM $\leftrightarrow$ disk



Examples

Typical states:

- **Active**: executing operations
- **Partially Committed**: last statement done
- **Committed**: effects made durable
- **Failed**: error/crash encountered
- **Aborted**: rolled back, effects undone

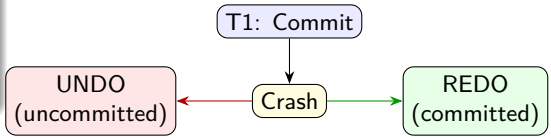


Failure Types

- **Transaction failure**: logic/constraint error
- **System crash**: OS/DBMS crash, power outage
- **Media failure**: disk corruption
- **Communication failure**: network loss

Effect

Some transactions are *uncommitted* at crash time $\Rightarrow$ must be **undone**. Committed ones must be **redone** if not fully flushed.



Log Records

- $\langle \text{BEGIN}, T_i \rangle$
- $\langle \text{UPDATE}, T_i, X, \text{old}, \text{new} \rangle$
- $\langle \text{COMMIT}, T_i \rangle$ or $\langle \text{ABORT}, T_i \rangle$

WAL Rule

Write-Ahead Logging: Log entries are flushed to stable storage *before* corresponding data pages are written to disk.

```
-- Abstract log illustration
<BEGIN T1>
<UPDATE T1, X, 100, 80>     -- X:
    100 -> 80
<UPDATE T1, Y, 50, 70>      -- Y:
    50 -> 70
<COMMIT T1>

-- Crash recovery:
-- REDO committed (T1), UNDO
   uncommitted
```

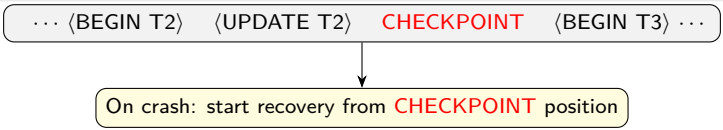
Idea

Periodically write a **checkpoint**:

- Flush dirty pages to disk
- Record active transactions
- Mark a point to start recovery scanning

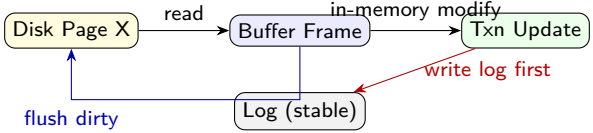
Examples

Recovery can start from the last checkpoint, avoiding a full log scan.



Buffer Manager Flow

- Read page from disk → **buffer frame** (RAM)
- Transaction updates page in buffer
- Log record flushed *before* dirty page (**WAL**)
- Background flush / checkpoint writes page to disk



Crash Safety

If crash occurs, **REDO** committed but not-yet-flushed updates; **UNDO** uncommitted ones.

Summary — Transaction & System Concepts

- **TM, CCM, RM, Buffer** = core system services enabling ACID.
- **States**: Active → Partially Committed → Committed / Failed → Aborted.
- **Failures**: decide REDO/UNDO using **Write-Ahead Log**.
- **Checkpoints** accelerate recovery; **buffers** optimize I/O safely under WAL.

Mental Model

"*Log before data, redo the committed, undo the rest, and use checkpoints to speed it up.*"

Table of Contents

Why Do We Need ACID?

Motivation

Transactions run concurrently and can fail unexpectedly. Without safeguards, we get:

- **Lost updates**, **dirty reads**, **inconsistent states**
- Partial effects when crashes happen in the middle

Goal

Guarantee correctness and reliability despite concurrency and failures.

Examples

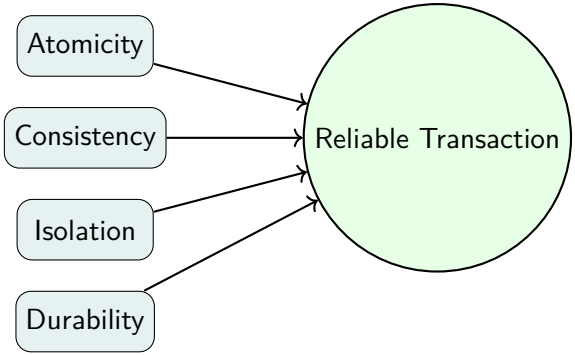
Real-world flows: bank transfer, e-commerce checkout, ticket booking

ACID

- **Atomicity**: All-or-none
- **Consistency**: Preserves constraints/invariants
- **Isolation**: No interference between concurrent transactions
- **Durability**: Once committed, effects persist

Without ACID

Dirty reads, lost updates, inconsistent totals, phantom rows, etc.



Intuition

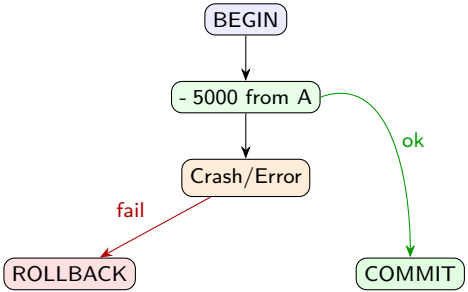
A transaction is an indivisible unit: either **all** its actions take effect or **none**.

Mechanism

Rollback (UNDO) using the log restores the database to the pre-transaction state if any step fails.

Examples

Bank transfer: if debit succeeds but credit fails, the system undoes the debit.



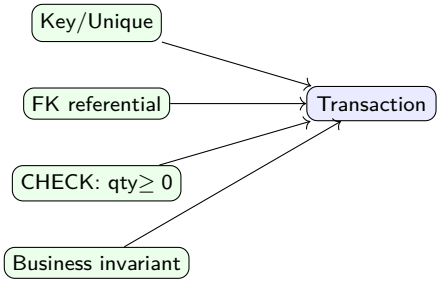
Meaning

Every transaction must move the database from one **valid** state to another, preserving:

- *Integrity constraints* (keys, FKs, checks)
- *Business rules* (e.g., non-negative balance)

Examples

- Total(A, B) remains constant after a transfer
- `qty` never below zero in inventory



Isolation — No Interference Between Transactions

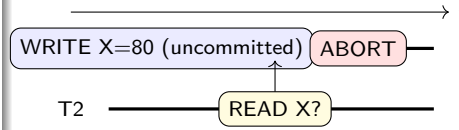
Meaning

Concurrent transactions behave as if executed **serially**. Prevent anomalies:

- **Dirty read**: read uncommitted data
- **Lost update**: one overwrites another's update
- **Unrepeatable read**: same query yields different results
- **Phantom**: new rows appear between scans

Mechanisms

Locking (2PL), **timestamp ordering**, **validation** protocols.



Dirty read if T2 sees 80 before T1 commits

Durability — Once Committed, Always There

Meaning

After a transaction **commits**, its effects **survive** system crashes.

Examples

Payment succeeds and receipt is shown; even if power fails immediately, the update is present after restart.

Mechanism

Write-Ahead Logging (WAL): *flush log first*, then data pages. On recovery, **REDO** committed work.

ACID at a Glance (Mechanisms & Examples)

Property	Guarantee	Ensured by
Atomicity	All-or-none	Undo/rollback via log
Consistency	Constraints preserved	Keys, FKs, checks, triggers
Isolation	Non-interference	2PL, timestamps, validation
Durability	Survive failures	WAL, checkpoints, stable storage

Examples

Steps:

1. Decrement stock
2. Create order
3. Charge payment
4. Confirm & notify

ACID ensures no half-order/half-payment, isolation from other orders, and durable confirmation.

```
BEGIN TRANSACTION;

UPDATE stock SET qty = qty - 1
WHERE sku='P123' AND qty > 0;

INSERT INTO orders(user_id, sku,
    status)
VALUES (42, 'P123', 'CREATED');

-- call payment; on success:
UPDATE orders SET status='CONFIRMED'
WHERE order_id=:oid;

COMMIT; -- else ROLLBACK on failure
```

Summary — ACID Properties

- **Atomicity**: all-or-none using UNDO
- **Consistency**: constraints always satisfied
- **Isolation**: concurrency without interference
- **Durability**: committed effects survive crashes

Key Insight

Log before data (WAL), redo committed, undo the rest — ACID makes transactions dependable.

Examples

Next: **Characterizing Schedules based on Recoverability & Serializability**.

Thank You!

Introduction to Schedule and Concurrency Control in DBMS

DBMS Lecture Series

Unit-6

Dr. Pavinder Yadav

Assistant Professor
School of Computer Science
University of Petroleum and Energy Studies
Dehradun

Outline

Table of Contents

What is a Schedule?

Definition

A **schedule** defines how operations of multiple transactions are interleaved during concurrent execution.

OR

It is chronological execution sequence of multiple transactions.

Examples

- If transactions execute one after another ⇒ **Serial Schedule**.
- If operations interleave ⇒ **Non-Serial Schedule**.

Goal

Maintain database **consistency** and **isolation** while improving concurrency.

Scenario

Two transactions operate on the same data item `A`, initially 100.

Transaction T1	Transaction T2
READ(A)	READ(A)
A = A - 50	A = A * 1.1
WRITE(A)	WRITE(A)

Goal

Observe how different schedules (serial vs. interleaved) yield different final results for `A`.

Case 1: Serial Schedule (T1 → T2)

Step	Operation	Value of A	Remarks
1	T1: READ(A)	100	Reads initial value
2	T1: A = A - 50	50	Deducts 50
3	T1: WRITE(A)	50	Updates A
4	T2: READ(A)	50	Reads new A
5	T2: A = A * 1.1	55	Adds 10%
6	T2: WRITE(A)	55	Final value

Result

Serial (T1 → T2): Final A = 55

Examples

Both transactions complete one after another — consistent state.

Case 2: Serial Schedule (T2 → T1)

Step	Operation	Value of A	Remarks
1	T2: READ(A)	100	Reads initial value
2	T2: A = A * 1.1	110	Adds 10%
3	T2: WRITE(A)	110	Updates A
4	T1: READ(A)	110	Reads updated A
5	T1: A = A - 50	60	Deducts 50
6	T1: WRITE(A)	60	Final value

Result

Serial (T2 → T1): Final A = 60

Examples

Another serial order — different but valid result.

Case 3: Non-Serial (Interleaved Schedule)

Step	Operation	Value of A	Remarks
1	T1: READ(A)	100	Reads initial value
2	T2: READ(A)	100	Reads same initial value
3	T1: A = A - 50	50	Local copy in T1
4	T2: A = A * 1.1	110	Local copy in T2
5	T1: WRITE(A)	50	Writes its result
6	T2: WRITE(A)	110	Overwrites A

Observation

Final A = 110 → Neither of the serial results (55 or 60). **Lost update:** T1's effect is overwritten by T2.

Comparison of All Schedules

Schedule	Execution Order	Final A	Serializable?
Case 1	T1 → T2	55	Yes
Case 2	T2 → T1	60	Yes
Case 3	Interleaved	110	No

Examples

Lost Update Problem: Both transactions read the same initial value and overwrite each other's changes.

Key Idea

Only serializable schedules guarantee consistency. DBMS enforces serializability using **locking** or **timestamp ordering**.

Classification

- **Serial:** Each transaction runs completely before the next begins.
- **Non-Serial:** Operations of different transactions are interleaved.
- **Serializable:** A non-serial schedule that yields the same result as a serial one.

Objective

Ensure concurrent execution is **equivalent to a serial order** — correctness with better performance.

Conflict and Non-Conflict Operation Pairs

Definition

Two operations conflict if:

1. They belong to **different transactions**,
2. They access the **same data item**, and
3. At least one of them is a **WRITE**.

Non-Conflict Pairs

$R(A)$ and $R(A)$
$R(B)$ and $R(A)$
$W(B)$ and $R(A)$
$R(B)$ and $W(A)$
$W(B)$ and $W(A)$

Conflict Pairs

$R(A)$ and $W(A)$
$W(A)$ and $R(A)$
$W(A)$ and $W(A)$

Examples

Conflicts occur only on the same data item (`A` or `B`) when one is a write.

Table of Contents

Motivation

Concurrent transactions may fail or rollback. We classify schedules by how easily they can recover from such failures.

Examples

- **Recoverable Schedule (RC)**
- **Cascadeless Schedule (CSC)**
- **Strict Schedule (SS)**

Definition

A schedule is **recoverable** if no transaction commits until all transactions from which it has read have committed. OR
A schedule is called **recoverable** if a transaction T2 that reads data written by another transaction T1 **commits only after** T1 has committed.

```
T1: WRITE(A)
T2: READ(A)-- reads value written by T1
T1: COMMIT
T2: COMMIT
```

T1	T2
W(A)	
	R(A)
Commit	
	Commit

Observation

T2 commits only after T1 commits → Safe recovery.

Examples

This ensures that if T1 fails or rolls back, T2 can also be rolled back safely without leaving inconsistent data.

What is a Non-Recoverable Schedule?

Definition

A schedule is **non-recoverable** if a transaction T2 commits **before** the transaction T1 whose data it has read commits.

Examples

If T1 later fails or rolls back, T2 has already committed using invalid data → inconsistency.

Step	Operation	Description	Status
1	T1: WRITE(A)	T1 modifies A	
2	T2: READ(A)	Reads value from T1 (uncommitted)	
3	T2: COMMIT	Commits early	Non-Recoverable
4	T1: ROLLBACK	Fails, undoes changes	

Effect

T2 is based on uncommitted data from T1 → system inconsistency.

Comparison: Recoverable vs Non-Recoverable

Aspect	Recoverable Schedule	Non-Recoverable Schedule
Commit Order	Dependent transactions commit after originals	Dependent transactions commit before originals
Safety on Failure	Safe — can rollback dependent transactions	Unsafe — inconsistent if original aborts
Read Data	From committed or uncommitted, but waits before commit	From uncommitted data
Allowed in DBMS	Allowed (safe)	Not allowed (unsafe)

Examples

DBMS ensures at least **recoverable schedules** — many systems go further to enforce **cascadeless** or **strict** schedules.

Key Idea

Recoverable = Safe to recover, Non-recoverable = Irreversible damage if crash occurs.

Definition

A schedule is **cascadeless** if every transaction reads only **committed data**.

```
T1: WRITE(A)
T2: READ(A)    -- must wait until T1 commits
T1: COMMIT
T2: READ(A)
T2: COMMIT
```

Key Idea

No transaction reads uncommitted data → no cascading rollbacks.

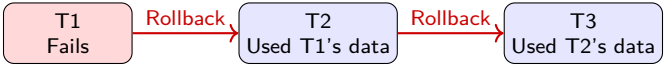
What is Cascading Rollback?

Definition

A **cascading rollback** happens when one transaction fails, and all other transactions that read its uncommitted data also need to rollback. A rollback in one transaction can cause others to rollback — called a **cascading rollback**.

Examples

Failure in one transaction **cascades** into others — like a chain reaction, wasted work and inconsistency risk.



Effect

Rollback of one transaction forces rollbacks of many others.

Situation

Transactions read data from uncommitted ones.

Step	Operation	Description	Status
1	T1: WRITE(A)	T1 updates A (uncommitted)	
2	T2: READ(A)	Reads uncommitted value from T1	Dependent
3	T3: READ(A)	Reads value indirectly via T2	Dependent
4	T1: ROLLBACK	T1 fails, undo its change	
5	—	T2 and T3 must rollback	Cascading Rollback

Observation

Rollback of T1 $\Rightarrow$ T2 and T3 also rollback $\rightarrow$ **Cascading Schedule**.

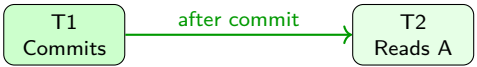
Definition

A **cascadeless schedule** ensures that a transaction reads data **only after** the transaction that wrote it has committed.

Examples

Prevents reading of uncommitted data → avoids chain rollbacks completely.

Step	Operation	Description	Status
1	T1: WRITE(A)	T1 updates A	
2	T1: COMMIT	T1 commits successfully	Safe
3	T2: READ(A)	Reads committed A	Safe
4	T2: COMMIT	Commits independently	Safe



Result

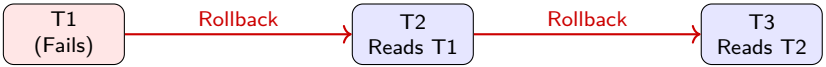
Each transaction depends only on **committed data**.

Cascading vs Cascadeless — Comparison

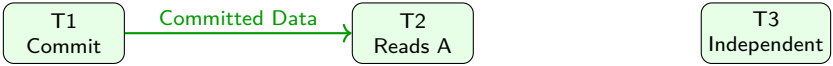
Aspect	Cascading Schedule	Cascadeless Schedule
Reads uncommitted data?	Yes, risky	No, only committed data
Can cause cascading rollback?	Yes, multiple rollbacks	No, isolated rollbacks
Recovery effort	High — affects many transactions	Low — only one transaction
Safety in DBMS	Unsafe	Safe
Performance	Faster but risky	Slightly slower, reliable

Examples

Cascadeless schedules are used in real DBMS (like MySQL, PostgreSQL) to guarantee safe recovery without chain rollbacks.



Cascading Rollback (Unsafe)



Cascadeless Schedule (Safe)

Definition

A schedule is **strict** if a transaction neither reads nor writes a data item until the last transaction that wrote it has committed or aborted.

```
T1: WRITE(A)
T2: (waits for T1 to commit)
T1: COMMIT
T2: READ(A)
T2: WRITE(A)
T2: COMMIT
```

Hierarchy

Strict $\subset$ Cascadeless $\subset$ Recoverable

Comparison of Schedule Types

Type	Reads uncommitted data?	Overwrites uncommitted data?	Rollback Effect
Recoverable	Possibly	No	Cascading rollback possible
Cascadeless	No	Possibly	Avoids cascading rollback
Strict	No	No	Simplest recovery

Inclusion Relation

Strict $\subset$ Cascadeless $\subset$ Recoverable

Why Recoverability Matters

Reasons

- Prevents committed transactions from depending on aborted ones.
- Avoids cascading rollbacks and wasted computation.
- Simplifies recovery after system or transaction failure.

Examples

Most commercial DBMS (like MySQL, PostgreSQL, Oracle) ensure **Strict Schedules** by default.

Summary — Recoverability of Schedules

- A recoverable schedule ensures dependent commits occur safely.
- A cascadeless schedule prevents reading uncommitted data.
- A strict schedule avoids both uncommitted reads and overwrites.

Hierarchy

Strict $\subset$ Cascadeless $\subset$ Recoverable

Examples

Next: **Characterizing Schedules Based on Serializability**.

Table of Contents

Why Serializability?

Motivation

Concurrent transactions improve throughput, but interleaving operations can cause incorrect results.

A schedule is **serializable** if its outcome is the same as some serial execution of the same transactions.

Examples

Ensures **correctness with concurrency**. Serializability is the **gold standard** for schedule correctness.

Example: Need for Serializability

Transaction T1	Transaction T2
READ(A)	READ(A)
A = A + 10	A = A * 2
WRITE(A)	WRITE(A)

Serial Order 1 (T1 → T2)

Initial A = 100 → Final A = 220

Serial Order 2 (T2 → T1)

Initial A = 100 → Final A = 210

Observation

Any interleaved schedule must give either 220 or 210 to be serializable.

Two main types

- **Conflict Serializability:** Based on order of conflicting operations.
- **View Serializability:** Based on who reads/writes which value.

Definition

A schedule is **conflict serializable** if it can be transformed into a serial schedule by swapping **non-conflicting operations**.

Goal

Allow interleaved execution that is **equivalent to a serial order**.

Definition

Two operations conflict if:

1. They belong to **different transactions**,
2. They access the **same data item**, and
3. At least one of them is a **WRITE**.

Non-Conflict Pairs

$R(A)$ and $R(A)$
$R(B)$ and $R(A)$
$W(B)$ and $R(A)$
$R(B)$ and $W(A)$
$W(B)$ and $W(A)$

Conflict Pairs

$R(A)$ and $W(A)$
$W(A)$ and $R(A)$
$W(A)$ and $W(A)$

Checking Conflict Equivalence Between Two Schedules

Definition

Two schedules S and S' are **conflict equivalent** if:

- They contain the same set of operations, and
- The order of all conflicting operations is identical in both.

Schedule S

T1	T2
R(A)	
W(A)	
	R(A)
	W(A)
R(B)	

Schedule S'

T1	T2
R(A)	
W(A)	
R(B)	
	R(A)
	W(A)

Question

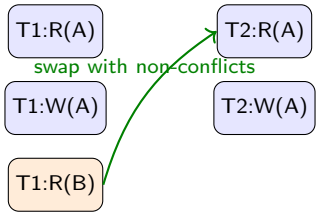
Are schedules S and S' **conflict equivalent**?

Idea

Two schedules are **conflict equivalent** if we can transform one into the other by repeatedly swapping **non-conflicting operations**.

Initial Schedule S:

T1	T2
R(A)	
W(A)	
	R(A)
	W(A)
R(B)	



After swapping non-conflicting operations:

T1	T2
R(A)	
W(A)	
R(B)	
	R(A)
	W(A)

Step 1: Identify that `R(B)` in T1 is on a **different data item** (B) than the operations on A $\rightarrow$ it is **non-conflicting**.

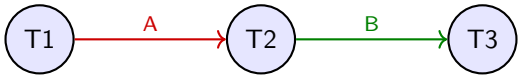
Result

Schedule S' obtained $\rightarrow$ order of all conflicting operations on A remains same. Since S and S' only differ by non-conflicting operations (on different data items), they are **conflict equivalent** in terms of final outcome.

Procedure

To test **Conflict Serializability**:

1. Create a node for each transaction.
2. Find all **conflicting operation pairs**.
3. Draw a directed edge $T_i \rightarrow T_j$ if T_i's operation comes before T_j's conflicting one.
4. If the graph is acyclic $\Rightarrow$ **Serializable**.



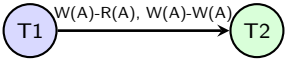
Examples

Arrows show the order of conflicting operations. If no cycle exists, schedule is conflict serializable.

Example 1: Conflict Serializable

```
Schedule S':
T1: READ(A)
T1: WRITE(A)
T2: READ(A)
T2: WRITE(A)
```

T1	T2
R(A)	
W(A)	
	R(A)
	W(A)



Conflicts

- T1.WRITE(A) → T2.READ(A) ⇒ **T1 → T2**
- T1.WRITE(A) → T2.WRITE(A) ⇒ **T1 → T2**

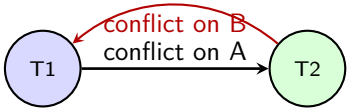
Result

Only edge T1 → T2, no cycle
⇒ **Schedule S is Conflict Serializable (Equivalent to serial order T1 → T2)**.

Example 2: Not Conflict Serializable

```
Schedule S:
T1: WRITE(A)
T2: READ(A)
T2: WRITE(B)
T1: READ(B)
```

T1	T2
W(A)	
	R(A)
	W(B)
R(B)	



Conflicts

- On A: T1.WRITE(A) $\rightarrow$ T2.READ(A) $\Rightarrow$ T1 $\rightarrow$ T2
- On B: T2.WRITE(B) $\rightarrow$ T1.READ(B) $\Rightarrow$ T2 $\rightarrow$ T1

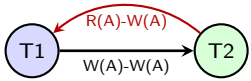
Observation

Cycle: T1 $\rightarrow$ T2 $\rightarrow$ T1 $\Rightarrow$ **Not conflict serializable**.

Example 3: Not Conflict Serializable

```
Schedule S:
T1: READ(A)
T2: READ(A)
T1: WRITE(A)
T2: WRITE(A)
```

T1	T2
R(A)	
	R(A)
W(A)	
	W(A)



Conflicts

- T2.READ(A) → T1.WRITE(A) ⇒ **T2 → T1**
- T1.WRITE(A) → T2.WRITE(A) ⇒ **T1 → T2**

Result

Cycle detected: T1 → T2 → T1
⇒ **Schedule S is Not Conflict Serializable**.

Example 4 — Acyclic Graph (Serializable)

```
Schedule S1:
T1: READ(A)
T1: WRITE(A)
T2: READ(A)
T2: WRITE(A)
```

Conflicts

- T1: WRITE(A) → T2: READ(A)
- T1: WRITE(A) → T2: WRITE(A)

Result

No cycle → Schedule is **conflict serializable** Equivalent serial order: (T1 → T2)

Example 5 — Cyclic Graph (Not Serializable)

```
Schedule S2:
T1: WRITE(A)
T2: READ(A)
T2: WRITE(B)
T1: READ(B)
```

Conflicts

- On A: T1 → T2
- On B: T2 → T1

Result

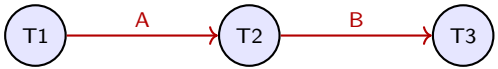
Cycle (T1 → T2 → T1) ⇒ **Not conflict serializable.**

Example 6 — Multi-Transaction Graph (Serializable)

```
Schedule S3:
T1: WRITE(A)
T2: READ(A)
T2: WRITE(B)
T3: READ(B)
T3: WRITE(C)
```

Conflicts

- T1 → T2 (on A)
- T2 → T3 (on B)



Result

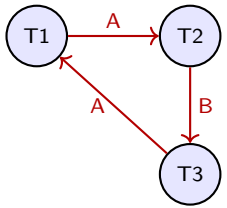
No cycle → Schedule is **conflict serializable** Equivalent serial order: (T1 → T2 → T3)

Example 7 — Three Transactions (Cyclic Case)

```
Schedule S4:
T1: WRITE(A)
T2: READ(A)
T2: WRITE(B)
T3: READ(B)
T3: WRITE(A)
```

Conflicts

- T1 → T2 (on A)
- T2 → T3 (on B)
- T3 → T1 (on A)



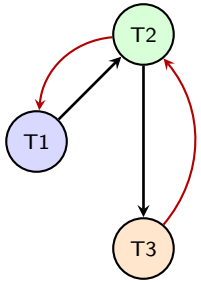
Result

Cycle (T1 → T2 → T3 → T1) → Schedule is **Not Conflict Serializable.**

Example: Conflict Serializability with Three Transactions

Schedule S:

T1	T2	T3
R(X)		
	R(Y)	
		R(Y)
	R(Z)	
		R(X)
R(Z)		
	W(X)	
W(Z)		
		W(Y)



Cycle present: $T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$

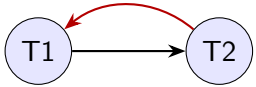
Result

Cycle exists $\Rightarrow$ **Schedule S is Not Conflict Serializable**.

Step	Operation	Transaction
1	READ(A)	T1
2	WRITE(A)	T2
3	READ(B)	T2
4	WRITE(B)	T1

Conflicts

- T1.READ(A) before T2.WRITE(A) $\Rightarrow$ T1 $\rightarrow$ T2
- T2.READ(B) before T1.WRITE(B) $\Rightarrow$ T2 $\rightarrow$ T1



Result

Cycle present $\rightarrow$ Not Conflict Serializable.

Definition

A schedule is **view serializable** if it preserves:

1. The same **read-from** relationships.
2. The same **final writes** on data items.

Examples

- Every **conflict-serializable** schedule is **view serializable**.
- Converse is not always true.

Example: View Serializability

```
T1: WRITE(A)
T2: WRITE(A)
T3: READ(A)
```

Observation

If T3 reads the same value of A as in some serial schedule, and the final write on A comes from the same transaction, then the schedule is **view serializable**.

Note

View serializability is broader but harder to check algorithmically.

Relation Between Schedule Types

Type	Basis	Condition
Conflict Serializable	Conflicting operations order	Acyclic precedence graph
View Serializable	Read-from, final-write equivalence	Logical equivalence check
Recoverable	Commit dependencies	Safe commit order

Hierarchy

Conflict Serializable $\subset$ View Serializable

Strict $\subset$ Cascadeless $\subset$ Recoverable

Implementation Approaches

- **Two-Phase Locking (2PL)** ensures conflict serializability.
- **Timestamp ordering** or **MVCC** used in modern systems (MySQL, PostgreSQL).
- Snapshot isolation approximates serializability with better performance.

Examples

Real systems favor **conflict serializability** for efficiency.

Summary — Serializability

- **Serializability** ensures concurrent execution correctness.
- **Conflict Serializability:** No cycles in precedence graph.
- **View Serializability:** Same reads/writes as some serial order.
- **Recoverable + Serializable** $\Rightarrow$ safe and consistent schedules.

Key Insight

"Acyclic graph = safe schedule; conflict serializable = practical guarantee of correctness."

Table of Contents

Why Concurrency Control is Needed

Motivation

Multiple transactions execute **concurrently** to improve performance and resource utilization. But without proper control, concurrency can lead to **inconsistent** results.

Examples

```
T1: Transfer 500 from A to B
T2: Check balance of A
```

If both run together, T2 may read intermediate (wrong) balance.

Goal

Maintain **Isolation** and **Consistency** while allowing parallel execution.

Problems Due to Uncontrolled Concurrency

Problem	Description	Example
Lost Update	One update overwrites another's result	T1 and T2 both change A=100 $\rightarrow$ final A=80
Dirty Read	Reading data modified but not committed yet	T2 reads A before T1 rolls back
Inconsistent Read	Reading partial results of another transaction	T1 reads balance while T2 updates it
Unrepeatable Read	Same query gives different results	T1 reads A twice, but T2 updates A in between
Phantom Read	Result set changes during execution	T1 reads salary >50k, T2 inserts new record

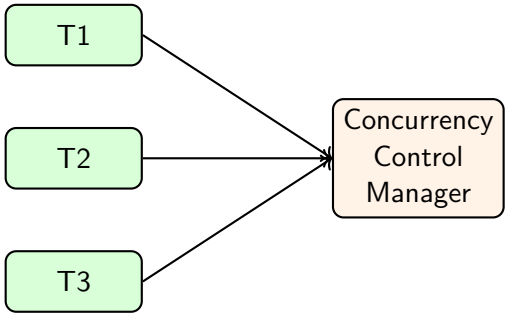
Observation

Concurrency improves performance, but without control it breaks **ACID properties**.

Definition

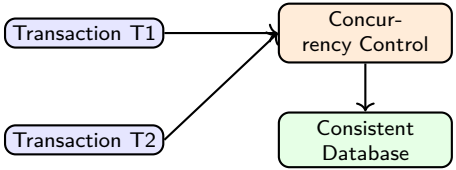
Concurrency Control is the mechanism that coordinates simultaneous transactions to preserve **database consistency and isolation**.

- Prevents conflicts among transactions.
- Ensures only valid interleavings are allowed.
- Maintains serializability and recoverability.



Main Objectives

- Maintain database **consistency**.
- Ensure **serializability** of concurrent transactions.
- Guarantee **isolation** (no intermediate interference).
- Support **recoverability** after failure.
- Prevent **deadlocks** and **starvation**.



Examples

The concurrency manager ensures multiple transactions behave as if executed serially.

Approaches to Concurrency Control

Common Techniques

- **Lock-based Protocols** (2PL)
- **Timestamp Ordering**
- **Validation (Optimistic) Control**
- **Multiversion Control (MVCC)**

Each Technique Ensures

- Correct serializable schedules
- Safe recovery from failures
- Maximum possible concurrency

Examples

DBMS chooses the most efficient method depending on workload. e.g., MySQL InnoDB uses **MVCC** (multiversion) and **2PL**.

Example Scenario

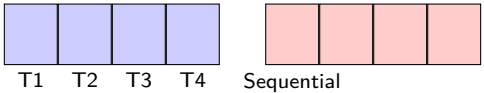
Without concurrency:
Each transaction = 2 seconds
10 transactions sequentially = 20 seconds.

Examples

With concurrency (4 transactions at once):
Total 6 seconds, same correctness, better utilization.

Key Point

Concurrency control improves throughput while ensuring that the result is same as a serial execution.



- Blue bars = concurrent (shorter time)
- Red bars = sequential (longer time)

Summary — Concurrency Control

- Concurrency allows multiple transactions to execute simultaneously.
- Without control $\rightarrow$ problems like lost update, dirty read, inconsistent read.
- Concurrency control ensures isolation, consistency, and recoverability.
- Techniques include:
  - Locking (2PL)
  - Timestamp ordering
  - Validation control
  - Multiversion concurrency control

Next Topic

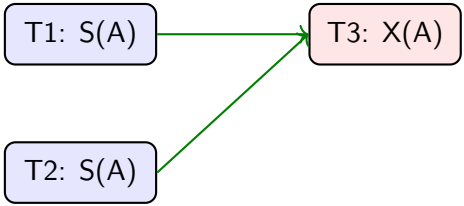
Two-Phase Locking (2PL) — Locking-based mechanism to ensure serializability.

Table of Contents

Definition

A **lock** is a control mechanism that restricts access to a data item by multiple transactions at the same time to prevent conflicts.

Lock Type	Purpose
Shared Lock (S)	Allows reading; multiple transactions can share it.
Exclusive Lock (X)	Allows writing; only one transaction can hold it.



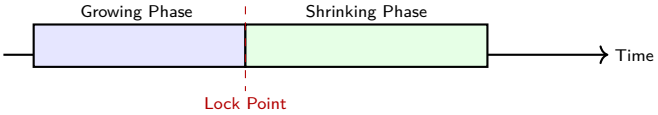
Rule

If a transaction holds an **X-lock**, no one else can acquire **S** or **X** lock on that item.

Definition

In **Two-Phase Locking**, every transaction has two phases:

1. **Growing Phase:** A transaction can acquire new locks on data items but cannot release any locks. It progressively locks the data it needs to access.
2. **Shrinking Phase:** Once the transaction releases its first lock, it enters the shrinking phase, during which it can release locks but cannot acquire any new ones.



Rule: Once a transaction releases even one lock, it **cannot acquire any new locks** afterwards.
How it works:

- When multiple transactions **run concurrently**, 2PL prevents them from interfering with each other by managing locks. This ensures that the final database state is **consistent**, as if the transactions had been executed serially.
- For a transaction to be serializable, it must follow the 2PL protocol, meaning all its lock requests must happen before its first lock is released.

Example of Two-Phase Locking (2PL)

Transactions

T1	Lock-X(A), Read(A), Write(A), Lock-X(B), Write(B), Unlock(A), Unlock(B)
T2	Lock-X(B), Read(B), Write(B), Lock-X(A), Write(A), Unlock(B), Unlock(A)

Step	Operation	Effect
1	T1: Lock-X(A)	Acquires A
2	T2: Lock-X(B)	Acquires B
3	T1: Lock-X(B)	Waits (T2 has B)
4	T2: Lock-X(A)	Waits (T1 has A)

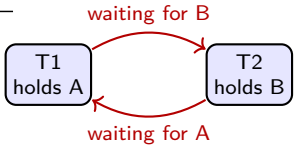
Observation

Both transactions wait for each other $\rightarrow$ **Deadlock**.

Condition

Deadlock occurs when each transaction holds one resource and waits for another.

Step	Operation	Effect
1	T1: Lock-X(A)	Acquires A
2	T2: Lock-X(B)	Acquires B
3	T1: Lock-X(B)	Waits (T2 has B)
4	T2: Lock-X(A)	Waits (T1 has A)



Examples

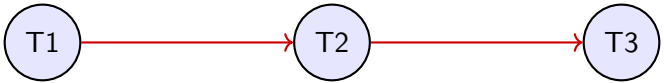
Detection: Wait-for graph with a cycle → deadlock exists.
Resolution: Abort one transaction and release its locks.

Handling Methods

- **Deadlock prevention:** order locks or timeout.
- **Deadlock detection:** use wait-for graph.
- **Deadlock recovery:** abort and restart one transaction.

Theorem

Every schedule produced by a transaction following 2PL is **conflict serializable**.



Examples

Because transactions cannot acquire locks after releasing any, the precedence graph is always **acyclic**, ensuring serializability.

Result

2PL $\rightarrow$ **Serializable but may cause deadlocks**.

Variants of Two-Phase Locking

Variant	Description	Deadlock?	Cascading Rollback?
Basic 2PL	Normal grow–shrink; release allowed after shrink begins	Possible	Possible
Strict 2PL	Holds all X-locks till commit/abort	Possible	No
Rigorous 2PL	Holds all locks (S and X) till commit	Possible	No
Conservative 2PL	Acquires all locks before execution begins	No	No

Observation

Strict 2PL is widely used as it prevents cascading rollbacks and simplifies recovery.

Without Locks

Step	Operation	Value of A
1	T1: Read(A=10)	10
2	T2: Read(A=10)	10
3	T1: A = A * 2	20
4	T1: Write(A=20)	20
5	T2: A = A + 5	15
6	T2: Write(A=15)	**Overwrites 20 → Error**

With 2PL Locks

Step	Operation	Value of A
1	T1: Lock-X(A), Read(A=10)	10
2	T1: A = A * 2, Write(A=20)	20
3	T1: Unlock(A)	20
4	T2: Lock-X(A), Read(A=20)	20
5	T2: A = A + 5, Write(A=25)	25
6	T2: Unlock(A)	25

Result

Without 2PL: Final A = 15 (Wrong)
With 2PL: Final A = 25 (Correct and Serializable)

Summary — Two-Phase Locking (2PL)

- Locks prevent conflicts between concurrent transactions.
- 2PL ensures serializability via growing and shrinking phases.
- Deadlocks are possible but manageable.
- Variants (Strict, Rigorous) simplify recovery.

Next Topic

Timestamp Ordering Concurrency Control — An alternative non-locking technique ensuring serializability using timestamps.

Motivation for Timestamp Ordering

Problem with Lock-based Methods

Locking (like 2PL) ensures serializability but may cause:

- **Deadlocks** (transactions waiting on each other)
- **Blocking Delays** (waiting for locks)
- **Overhead** for maintaining locks

Solution Idea

Instead of locking data, we can **order transactions by timestamps**.
Each transaction gets a unique timestamp when it starts.

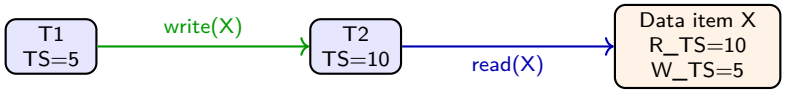
Examples

If $TS(T_1) = 5$ and $TS(T_2) = 10$, system ensures all actions of T_1 appear before T_2 in the final schedule.

Definition

Each transaction is assigned a unique, increasing **timestamp (TS)** when it starts.

- $TS(T_i)$ = Timestamp of transaction T_i
- Each data item X keeps two timestamps:
  - $R\_TS(X)$ — largest TS of any transaction that read X
  - $W\_TS(X)$ — largest TS of any transaction that wrote X



Examples

System guarantees: actions occur in the same order as timestamps.

Read Rule

When T_i issues `READ(X)`:

- If $TS(T_i) < W\_TS(X) \rightarrow$ Younger transaction has already written X $\rightarrow$ **Abort** T_i
- Else $\rightarrow$ Perform read $\rightarrow$ $R\_TS(X) = \max(R\_TS(X), TS(T_i))$

Write Rule

When T_i issues `WRITE(X)`:

- If $TS(T_i) < R\_TS(X) \rightarrow$ Abort T_i
- If $TS(T_i) < W\_TS(X) \rightarrow$ Abort T_i
- Else $\rightarrow$ Perform write $\rightarrow$ $W\_TS(X) = TS(T_i)$

Examples

The rules guarantee conflict-serializable order according to timestamps.

Example 1: Timestamp Ordering Protocol

Given

- Transactions with timestamps: $TS(T_1) = 100, \quad TS(T_2) = 200, \quad TS(T_3) = 300$
- Data items: A, B, C
- Initial timestamps: $RTS(A) = RTS(B) = RTS(C) = 0, \quad WTS(A) = WTS(B) = WTS(C) = 0$

Schedule:

T1	T2	T3
R(A)		
	R(B)	
W(C)		
		R(B)
R(C)		
	W(B)	
		W(A)

Rules:

1. **Read Rule:**
   - If $WTS(X) > TS(T_i) \rightarrow$ Rollback T_i
   - Else $\rightarrow$ execute Read(X)
   - Update: $RTS(X) = \max(RTS(X), TS(T_i))$
2. **Write Rule:**
   - If $RTS(X) > TS(T_i) \rightarrow$ Rollback T_i
   - If $WTS(X) > TS(T_i) \rightarrow$ Rollback T_i
   - Else $\rightarrow$ perform Write(X); set $WTS(X) = TS(T_i)$

Execution and Timestamp Table

Step	Operation	Condition Check	Action	RTS(A,B,C)	WTS(A,B,C)
1	T1: R(A)	$WTS(A) = 0 < TS(100)$	OK, Read A	(100,0,0)	(0,0,0)
2	T2: R(B)	$WTS(B) = 0 < TS(200)$	OK, Read B	(100,200,0)	(0,0,0)
3	T1: W(C)	$RTS(C) = 0 < TS(100)$	OK, Write C	(100,200,0)	(0,0,100)
4	T3: R(B)	$WTS(B) = 0 < TS(300)$	OK, Read B	(100,300,0)	(0,0,100)
5	T1: R(C)	$WTS(C) = 100 < TS(100)$	OK, Read C	(100,300,100)	(0,0,100)
6	T2: W(B)	$RTS(B) = 300 > TS(200)$	Abort T2	(100,300,100)	(0,0,100)
7	T3: W(A)	$RTS(A) = 100 < TS(300)$	OK, Write A	(100,300,100)	(300,0,100)

Explanation

- $T2$ aborts because a younger transaction ($T3$) has already read B before $T2$ tried to write it.
- $T3$ successfully writes A as no conflicts violate timestamp order.

Final Timestamp Values

$$RTS(A, B, C) = (100, 300, 0)$$

$$WTS(A, B, C) = (300, 0, 100)$$

Examples

- $T1$ (oldest) $\rightarrow$ completed successfully.
- $T2 \rightarrow$ Aborted due to violation (younger read before older write).
- $T3$ (youngest) $\rightarrow$ executed successfully.

Conclusion

The final serial order = $T1 \rightarrow T3$ (since $T2$ was rolled back). **Timestamp ordering** ensures conflict-serializable schedule.

Example 1 — Simple Case (Serializable)

Transaction	Timestamp	Operations
T1	5	Read(X), Write(X)
T2	10	Read(X)

Step	Operation	Rule Checked	Result	Updated Timestamps
1	T1: Write(X)	$W\_TS(X) = 0$	Allowed	$W\_TS(X) = 5$
2	T2: Read(X)	$TS(10) > W\_TS(5)$	Allowed	$R\_TS(X) = 10$

Examples

Schedule consistent with timestamp order (T1 before T2). → **Conflict Serializable**.

Example 2 — Violation Case (Abort)

Transaction	Timestamp	Operations
T1	10	Read(X)
T2	5	Write(X)

Step	Operation	Rule Checked	Result	Updated TS
1	T1: Read(X)	Allowed	$R\_TS(X) = 10$	
2	T2: Write(X)	$TS(5) < R\_TS(10)$	Abort T2	

Explanation

T2 (older transaction) tries to write a value that a newer transaction (T1) has already read. → Violates timestamp order. → **T2 is rolled back**.

Transaction	Timestamp	Operations
T1	5	Write(X)
T2	10	Write(X)

Step	Operation	Check	Result	Updated TS
1	T1: Write(X)	Allowed	$W\_TS(X) = 5$	
2	T2: Write(X)	$TS(10) > W\_TS(5)$	Allowed	$W\_TS(X) = 10$

Examples

Both writes are allowed; final write belongs to T2 (newer timestamp). Schedule follows timestamp order → **Serializable**.

Advantages and Disadvantages of TOCC

Advantages

- No deadlocks (no waiting for locks)
- Ensures serializability naturally
- Simpler to reason than 2PL

Disadvantages

- Frequent **aborts** when timestamps conflict
- Requires extra storage for timestamps
- May waste work of long transactions

Examples

Timestamp ordering is suitable for read-heavy or low-conflict workloads.

Variant	Description
Basic TOCC	Follows basic read/write timestamp rules.
Strict TOCC	Delays write until commit; avoids cascading rollbacks.
Multiversion TOCC (MVTO)	Keeps multiple versions of each item, one per write timestamp.

Observation

Multiversion TOCC (used in **MVCC** in MySQL/PostgreSQL) allows readers to read older versions → eliminates read–write conflicts.

Given:

Initial value of X = 100

- T1 (TS=5): Write(X=150)
- T2 (TS=10): Read(X)

Step	Operation	W_TS(X)	R_TS(X)	Allowed?
1	T1: Write(X=150)	5	0	Yes, update W_TS=5
2	T2: Read(X)	5	0	Yes, update R_TS=10
3	T1: Commit	—	—	Serializable (T1 before T2)

Examples

Final value X = 150 → consistent with order (T1 → T2).

Result

Timestamp Ordering enforces serializability without locks!

Summary — Timestamp Ordering Concurrency Control

- Each transaction assigned a unique timestamp.
- All reads and writes executed in timestamp order.
- Violations $\rightarrow$ abort (no waiting, no deadlocks).
- Ensures serializability without locks.

Next Topic

Validation (Optimistic) Concurrency Control Techniques — based on validating transactions just before commit.

Motivation for Optimistic Concurrency Control (OCC)

Observation

In many systems, most transactions are **read-only** or **non-conflicting**. Using locks or timestamps for all of them is unnecessary and slows down the system.

Idea

Let transactions execute freely (no locks). At the end, before commit, **validate** that no conflicts occurred.

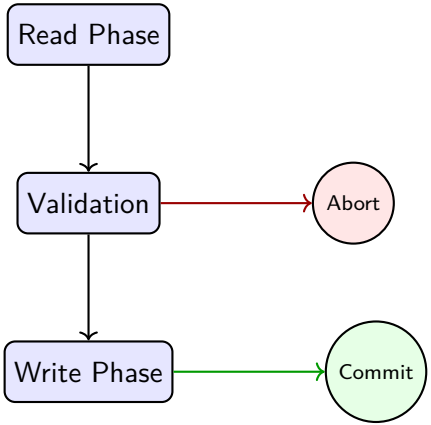
Examples

Called **Optimistic** because it assumes:

"Conflicts are rare — let everyone work independently and check later."

Three Phases of OCC

1. **Read Phase:**
   - Transaction reads data and performs computation.
   - Updates are made on a **local copy**.
2. **Validation Phase:**
   - System checks for conflicts with other committed or validating transactions.
3. **Write Phase:**
   - If validation passes → updates are written to database.
   - Otherwise → transaction **aborts** and restarts.



Timestamps

Each transaction T_i has:

- $Start(T_i)$ — when it begins.
- $Validation(T_i)$ — when validation starts.
- $Finish(T_i)$ — when it commits.

Validation Rule

For every pair of transactions T_i and T_j:

- If T_i completes before T_j starts $\rightarrow$ no conflict.
- If they overlap, check read and write sets:

$$Write\_set(T_i) \cap Read\_set(T_j) = \emptyset$$

safe, otherwise conflict $\rightarrow$ abort one.

Examples

Example 1 — No Conflict

Transaction	Operations	Data Items
T1	Read(A), Compute(A×2), Write(A)	A
T2	Read(B), Compute(B+10), Write(B)	B

Validation

- $Write\_set(T1) = \{A\}$
- $Write\_set(T2) = \{B\}$
- $Write\_set(T1) \cap Write\_set(T2) = \emptyset$

Result

No overlapping items → both transactions can commit safely. Schedule is **Serializable**.

Example 2 — Conflict Detected

Transaction	Operations	Data Items
T1	Read(X), Compute, Write(X)	X
T2	Read(X), Compute, Write(X)	X

Step	Event	Validation Check	Result
1	T1 reads X=50	—	OK
2	T2 reads X=50	—	OK
3	T1 writes X=60	—	OK
4	T2 validates	$Write(T1) \cap Read(T2) = \{X\}$	Abort T2

Explanation

T2 read X before T1 finished, but T1 later wrote X. → Conflict detected during validation → T2 must abort.

Step	Transaction	Action	X Value	Validation Result
1	T1	Read(X=10)	10	—
2	T2	Read(X=10)	10	—
3	T1	Write(X=20)	20	—
4	T1	Commit	20	—
5	T2	Validate	X modified by T1	Abort

Result

$T2$ must restart using the updated X=20 $\rightarrow$ maintains serializability.

Advantages and Disadvantages of OCC

Advantages

- No locks, no deadlocks.
- High degree of concurrency.
- Ideal for read-heavy workloads.

Disadvantages

- High abort rate under contention.
- Extra validation overhead.
- Poor for write-intensive environments.

Examples

OCC is widely used in distributed or analytical databases where conflicts are rare.

Analogy

Multiple users edit different parts of a report offline (no locks). Before submission, system checks if anyone edited the same section.

- If yes → conflict → one user's work is rejected (abort).
- If no → both commits (merge successful).

Summary

- OCC uses **Read → Validate → Write** phases.
- Transactions are validated before commit for serializability.
- Works best when conflicts are **rare**.
- Provides **maximum parallelism** and **no deadlocks**.

Table of Contents

Motivation for Multiple Granularity Locking

Problem with Simple Locking

- Locking entire database → very low concurrency.
- Locking each record → too many locks, high overhead.

Need

A mechanism to lock at **different levels of data** efficiently.
This is achieved using **Multiple Granularity Locking (MGL)**.

Examples

Example:

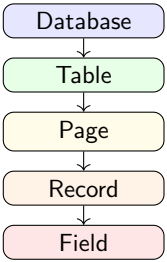
- T1 reads entire `Employee` table.
- T2 updates one record of `Employee`.

Both can coexist if system manages locks properly at multiple levels.

Definition

Granularity refers to the **size of data item** that can be locked.

Level	Example
Database	Entire DB instance
Table	e.g., Employee table
Page	Disk block containing several records
Record/Tuple	Single row of a table
Field	One attribute of a row



Examples

DBMS maintains this hierarchy for efficient lock management.

Concept of Multiple Granularity Locking (MGL)

Definition

MGL allows transactions to lock **different levels of data items** in a hierarchy simultaneously, ensuring safe coexistence of locks.

Key Idea

Use **Intention Locks** to indicate a transaction's plan to obtain finer-granularity locks at lower levels.

Examples

Example Hierarchy:

$$\text{Database} \rightarrow \text{Table} \rightarrow \text{Page} \rightarrow \text{Record}$$

If a transaction wants to modify one record, it first acquires intention locks on its ancestors.

Lock Type	Meaning
S (Shared)	Transaction reads the data item. Multiple S-locks allowed.
X (Exclusive)	Transaction reads and writes the data item. Only one allowed.
IS (Intention Shared)	Intends to acquire S-locks on lower-level items.
IX (Intention Exclusive)	Intends to acquire X-locks on lower-level items.
SIX (Shared + Intention Exclusive)	Reads all items but may modify a few.

Examples

Example: To X-lock a record, a transaction must hold:

$$IS(DB) \Rightarrow IX(Table) \Rightarrow X(Record)$$

Lock Compatibility Matrix

Held ↓ / Requested →	IS	IX	S	SIX	X
IS	✓	✓	✓	✓	×
IX	✓	✓	×	×	×
S	✓	×	✓	×	×
SIX	✓	×	×	×	×
X	×	×	×	×	×

Interpretation

- ✓→ Locks can coexist safely.
- ×→ Locks conflict; one transaction must wait.

Examples

IX vs S → conflict (T1 writing, T2 reading). IS vs S → compatible (both intend to read).

Example — Hierarchical Locking Scenario

Hierarchy:

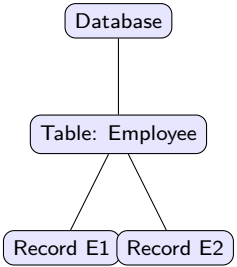
$$Database \rightarrow Table(Employee) \rightarrow Record(E1)$$

Transactions:

- T1 updates Record E1.
- T2 reads entire Employee table.

Locks Acquired:

Level	T1	T2
Database	IS	IS
Table (Employee)	IX	S
Record (E1)	X	—



Conflict

IX (T1) vs S (T2) → Conflict at Table level. One must wait → ensures serializability.

Scenario

Transaction T3 reads the entire Employee table and updates a few records.

Level	Lock Type	Meaning
Table (Employee)	SIX	Shared + Intention Exclusive
Record (E2)	X	Updates a record
Record (E4)	X	Updates another record

Result

T3 can:

- Read all records (due to S part).
- Write some records (due to IX part).
- Block other writers but allow other readers at non-conflicting levels.

Locking Rules

1. Before locking an item, a transaction must first obtain **intention locks** on all its ancestors.
2. Locks must be acquired in a **top-down** manner.
3. Locks must be released in a **bottom-up** manner.
4. Often combined with **2PL** to ensure serializability.

Examples

To X-lock Record R:

$$\text{IS(Database)} \rightarrow \text{IX(Table)} \rightarrow \text{X(Record)}$$

Goal

Safe hierarchical locking without unnecessary blocking or conflicts.

Advantages and Disadvantages of MGL

Advantages

- Increases concurrency by mixed lock levels.
- Reduces number of locks needed.
- Balances overhead and performance.

Disadvantages

- More complex to implement.
- Compatibility matrix overhead.
- Deadlocks still possible if rules not followed.

Examples

MGL is used in modern DBMSs (e.g., SQL Server, Oracle) to lock tables, pages, and rows efficiently.

Summary — Multiple Granularity Locking

- **Granularity** defines the size of data items that can be locked.
- **Multiple Granularity Locking (MGL)** allows locks at various levels.
- Uses **Intention Locks (IS, IX, SIX)** for safe hierarchical control.
- Locking follows **top-down** and unlocking **bottom-up**.
- Achieves efficient concurrency with fewer locks.

Thank You!

Introduction to Recovery Concepts in DBMS

DBMS Lecture Series

Unit-6

Dr. Pavinder Yadav

Assistant Professor
School of Computer Science
University of Petroleum and Energy Studies
Dehradun

Outline

Table of Contents

Why Recovery is Needed

Even with concurrency control, failures can occur due to:

- **System Crash:** Power failure, OS crash.
- **Transaction Failure:** Logical error or invalid operation.
- **Media Failure:** Disk head crash or corruption.
- **Communication Failure:** Loss in distributed networks.

Goal of Recovery

Ensure that after any failure:

- Database is restored to a **consistent state**.
- No committed transaction is lost.

Examples

Example: Power fails while transferring money from A to B — recovery ensures no partial update.

ACID Recap

- **Atomicity:** Transaction executes completely or not at all.
- **Consistency:** Database constraints remain valid.
- **Isolation:** Intermediate results not visible to others.
- **Durability:** Once committed, results are permanent.

Recovery Ensures

- **Atomicity** through UNDO operations.
- **Durability** through REDO operations.

Examples

If a crash occurs after commit $\rightarrow$ REDO the transaction. If before commit $\rightarrow$ UNDO it.

What is a Log File?

A sequential record of all database modifications used for recovery.

Log Record Type	Description
<T_i, start>	Transaction T_i started.
<T_i, X, old, new>	T_i updated data item X: old $\rightarrow$ new.
<T_i, commit>	Transaction committed successfully.
<T_i, abort>	Transaction aborted/rolled back.

Write-Ahead Logging (WAL)

Log record must be written to disk **before** the actual data page is updated.

Example: Log Records Before Crash

```
<T1, start>
<T1, A, 50, 60>
<T1, commit>
<T2, start>
<T2, B, 100, 150>
---- System Crash ----
```

Recovery Process

- T1 has commit record → **REDO**.
- T2 has no commit record → **UNDO**.

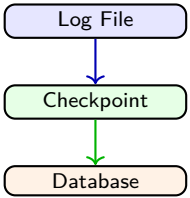
T1 Commit → REDO ⟶ Redo Changes

T2 Abort → UNDO ⟶ Undo Changes

Motivation

Without checkpoints, system must scan the entire log after a crash.
Checkpoints help restart recovery from a known consistent point.

1. Stop accepting new transactions.
2. Write all modified (dirty) buffers to disk.
3. Write a log record: `<checkpoint, active_Txns>`.
4. Resume normal operation.



Examples

If a crash occurs, restart recovery from the **last checkpoint**, not from the beginning of the log.

Components of Recovery Manager

Component	Responsibility
Transaction Manager	Maintains transaction states (active, committed, failed).
Log Manager	Writes log records and ensures WAL rule.
Checkpoint Manager	Periodically stores consistent DB snapshots.
Buffer Manager	Transfers data between memory and disk.
Recovery Manager	Performs REDO and UNDO during system restart.

Interaction

All managers cooperate to maintain the **ACID** guarantees of the DBMS.

Examples

Example: When crash occurs, Log Manager provides info, and Recovery Manager executes actions.

Summary — Recovery Concepts

- Recovery ensures **Atomicity** and **Durability**.
- Uses **log-based** recovery to maintain change history.
- **Write-Ahead Logging (WAL)** $\rightarrow$ log before data.
- **Checkpoints** reduce recovery time.
- Recovery Manager performs:
  - **UNDO** for uncommitted transactions.
  - **REDO** for committed ones.

Next Topic

Recovery Techniques Based on Deferred and Immediate Update

Table of Contents

Two Main Approaches

Based on **when** the updates are written to the database:

1. **Deferred Update** — changes are delayed until commit.
2. **Immediate Update** — changes are written immediately.

Common Foundation

Both techniques depend on **Write-Ahead Logging (WAL)**:

Log record must be written before the actual data is updated.

Examples

Deferred Update → safer, simpler. Immediate Update → faster but needs both UNDO and REDO.

Deferred Update (No-Undo / Redo)

Concept

In **Deferred Update**, database is not updated until the transaction commits.

- All updates are stored in the log or local buffer.
- Once the transaction commits, the changes are written to the database.
- If crash occurs before commit $\rightarrow$ ignore the transaction.

Hence

Only **REDO** is needed during recovery. This is also called **NO-UNDO / REDO** method.

Examples

If T1 commits successfully $\rightarrow$ redo its changes. If T2 fails before commit $\rightarrow$ no action needed.

Step	Log Entry	Database Action
1	`<T1, start>`	—
2	`<T1, A, 200>`	Kept in log (no DB update)
3	`<T1, B, 80>`	Kept in log (no DB update)
4	`<T1, commit>`	Apply updates to DB: A=200, B=80

Crash Handling

- If crash occurs before Step 4 → nothing written → no undo required.
- If crash occurs after Step 4 → redo changes from log.

Examples

Only committed transactions are redone after recovery.

Advantages and Disadvantages of Deferred Update

Advantages

- Simple recovery mechanism.
- No need for undo operation.
- Safe from partial updates or crash during execution.

Disadvantages

- Slower commit (all writes at once).
- Requires larger log buffer.
- Delayed data visibility.

Examples

Used where durability and correctness are more important than speed.

Immediate Update (Undo / Redo)

Concept

In **Immediate Update**, database is updated as soon as the operation executes, even before transaction commits.

Therefore

- Both **UNDO** (for uncommitted transactions) and **REDO** (for committed ones) are required.
- Log entries must include both old and new values.

Examples

If system fails after partial updates → Uncommitted changes are undone, committed ones are redone.

Step	Log Entry	Database Action
1	`<T2, start>`	—
2	`<T2, A, 100, 200>`	A updated immediately in DB
3	`<T2, B, 50, 70>`	B updated immediately in DB
4	`---- Crash before commit ----`	Rollback using old values

Crash Handling

- If transaction not committed → use **old values** from log to undo.
- If committed → use **new values** to redo after restart.

Examples

Both undo and redo operations may occur during recovery.

Comparison: Deferred vs Immediate Update

Aspect	Deferred Update	Immediate Update
When DB updated	After commit	During execution
Recovery type	REDO only (No Undo)	UNDO + REDO
Log record	New values only	Old + new values
Commit speed	Slower (writes delayed)	Faster (writes immediate)
Safety	High (no partial updates)	Requires WAL for safety
Complexity	Simple	More complex
Typical use	Banking or batch systems	Real-time systems

Key Difference

Deferred → safer, simpler; Immediate → faster, requires undo/redo.

Summary — Deferred and Immediate Update Techniques

- **Deferred Update:**
  - Updates applied only after commit.
  - Requires REDO only.
  - No risk of partial updates.
- **Immediate Update:**
  - Updates written immediately.
  - Needs both UNDO and REDO.
  - Requires careful logging.

Next Topic

Shadow Paging — a recovery method without logs, using page-level shadow copies.

Table of Contents

Concept

Shadow Paging is a recovery technique that **does not use logs**.

- Maintains two copies of the database:
  - **Current Copy** — modified during transaction.
  - **Shadow Copy** — last committed stable version.
- If crash occurs $\rightarrow$ revert to shadow copy.
- If transaction commits $\rightarrow$ shadow copy replaced by new one.

Goal

Ensure **Atomicity and Durability** without using UNDO or REDO.

Examples

Changes made by a transaction are visible only after commit.

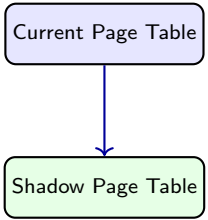
Page Tables

DBMS divides the database into fixed-size **pages** and maintains:

- **Current Page Table (CPT):** Used during transaction execution.
- **Shadow Page Table (SPT):** Saved stable copy for recovery.

Example Mapping

Logical Page	Physical Page (Disk)
P1	101
P2	102
P3	103



Examples

SPT represents the last consistent version of the database.

Steps in Shadow Paging

Algorithm

1. Create a copy of the **Shadow Page Table** when transaction starts.
2. For each updated page:
   - Write it to a new disk location (do not overwrite original page).
   - Update mapping in the **Current Page Table**.
3. On **commit:** replace shadow page table with the current one.
4. On **abort or crash:** discard current page table, keep shadow copy.

Recovery Property

- No need for log-based UNDO or REDO.
- Always maintain a consistent shadow version on disk.

Example — Page Table Switching

Initial State:

Page	Physical Block
P1	101
P2	102
P3	103

After T1 modifies P1:

Page	Physical Block
P1	201 (new copy)
P2	102
P3	103

Examples

On commit → replace shadow table with current one. On crash → discard current, use shadow table.

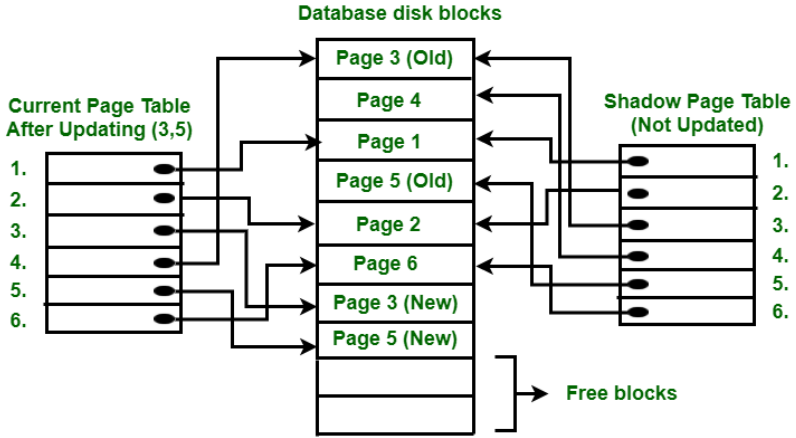
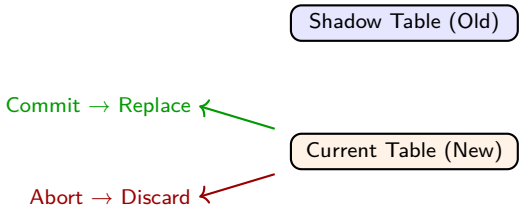


Figure: Shadow Paging

Recovery Rules

- If **transaction commits:** Replace shadow page table with current one.
- If **transaction aborts/crashes:** Discard current page table, revert to shadow.
- No need for log scanning, redo, or undo.



Examples

Shadow Paging provides instant recovery by switching page tables atomically.

Advantages and Disadvantages of Shadow Paging

Advantages

- Fast and simple recovery — no logging.
- Always keeps a consistent version on disk.
- No UNDO or REDO required.
- Excellent for single-user or read-only systems.

Disadvantages

- Copying page tables each time → expensive.
- Inefficient for large databases.
- Poor concurrency support.
- Double storage required temporarily.

Examples

Often used in file systems (e.g., NTFS) and small embedded DBs.

Summary — Shadow Paging

- Recovery technique that maintains **two copies** of data: current and shadow.
- Updates occur on new pages — no overwriting.
- On commit $\rightarrow$ shadow replaced with new copy.
- On crash $\rightarrow$ revert to old shadow copy.
- **No logs, no undo, no redo** — atomic switch ensures consistency.

Key Takeaway

Shadow Paging ensures instant recovery with zero log overhead, but is costly for large, multi-user environments.

Thank You!